

## 本次新增算法补丁

本补丁供已打印模板临时增页：Radix Heap 插入第 3 章 3.5 末尾，Borůvka 插入第 5 章 5.5 末尾。代码分别沿用对应章节公共前导；Borůvka 依赖 5.5.001 的 DSU 与 MEdge。

### 3.5.013 Radix Heap / 基数堆

Radix Heap 是键单调的整数最小优先队列：每次新插入的键都必须不小于最近一次弹出的键。它把键按“与最近弹出键 `last` 的最高不同位”分到 65 个桶；当第 0 桶为空时，找到首个非空桶，以桶内最小键更新 `last`，再把整桶重新分配。更新后该最小键必进入第 0 桶，而其余元素进入编号更小的桶，所以每个元素至多被重新分配 64 次。这个前提正好适用于非负边权 Dijkstra 等松弛只会产生不小于当前距离的新键的算法；普通可减小键、负权或任意顺序优先队列不能使用本结构。

```
1  template <class T> class RadixHeap
2  {
3      using Key = unsigned long long; // 键依赖 64 位无符号异或与位宽。
4      // b[i] 保存与 last 的最高不同位落在第 i 桶的键值对。
5      array<vector<pair<Key, T>>, 65> b;
6      // last 是最近弹出的键, n 是当前元素数。
7      Key last = 0;
8      int n = 0;
9
10     int id(Key x) const
11     {
12         // x 是不小于 last 的键; 返回它当前所属的桶编号。
13         return x == last ? 0 : bit_width(x ^ last);
14     }
15
16     void pull()
17     {
18         // 无参数; 把首个非空高桶按其中最小键重新分桶, 使 0 号桶非空。
19         if (!b[0].empty())
20         {
21             return;
22         }
23         int p = 1;
24         while (b[p].empty())
25         {
26             p++;
27         }
28         last = min_element(b[p].begin(), b[p].end(),
29                             [](const auto &x, const auto &y)
30                             {
31                                 return x.first < y.first;
32                             })->first;
33         for (auto x : b[p])
34         {
35             b[id(x.first)].push_back(move(x));
36         }
```

```

37     b[p].clear();
38 }
39
40 public:
41     bool empty() const
42     {
43         // 无参数; 返回堆是否为空。
44         return n == 0;
45     }
46
47     int size() const
48     {
49         // 无参数; 返回当前元素数。
50         return n;
51     }
52
53     void push(Key k, const T &v)
54     {
55         // k 是单调键, v 是关联值; 插入一项, 要求 k 不小于最近弹出的键。
56         assert(k >= last); // 调试检查, 可删
57         b[id(k)].push_back({k, v});
58         n++;
59     }
60
61     const pair<Key, T> &top()
62     {
63         // 无参数; 返回键最小的一项, 要求堆非空。
64         assert(n > 0); // 调试检查, 可删
65         pull();
66         return b[0].back();
67     }
68
69     pair<Key, T> pop()
70     {
71         // 无参数; 删除并返回键最小的一项, 要求堆非空。
72         auto ans = top();
73         b[0].pop_back();
74         n--;
75         return ans;
76     }
77 };

```

构造 `RadixHeap<T> q` 后, `push(k,v)` 插入键值对, `top()` 只读一个最小键元素, `pop()` 删除并返回一个最小键元素; 同键元素之间没有稳定顺序。 `top` 返回的引用会在下一次修改操作后失效。 键必须是  $[0, 2^{64})$  内的整数, 并始终满足  $k \geq \text{last}$ ; 这里保留 `unsigned long long` 是因为分桶确实依赖精确的 64 位无符号异或和位宽。 `push` 为  $O(1)$ , 每个元素一生最多移动 64 次, 故  $n$  次插入、弹出共  $O(64n)$  时间、 $O(n + 64)$  空间; 若把键宽记为  $W$ , 则为  $O(Wn)$ 。 不满足单调键时使用标准 `priority_queue`; 需要可并堆时使用 3.5.008 左偏树 / 可并堆。

下面检查重复键、跨高位键和单调弹出:

```
1 signed main()
2 {
3     RadixHeap<int> q;
4     q.push(7, 1);
5     q.push(3, 2);
6     q.push(3, 3);
7     q.push(1ULL << 40, 4);
8     assert(q.pop().first == 3);
9     assert(q.pop().first == 3);
10    q.push(8, 5);
11    assert(q.pop().first == 7);
12    assert(q.pop().first == 8);
13    assert(q.pop().first == (1ULL << 40) && q.empty());
14 }
```

## 5.5.014 Borůvka

Borůvka 算法维护当前最小生成森林，并在每一轮为每个连通块选择一条权值最小的出边。割性质保证这些边都可以安全加入某棵最小生成树；合并所有仍连接不同块的候选边后，连通块数至少减半，因此最多进行  $O(\log n)$  轮。它不需要预排序，适合边按批次扫描、并行寻找各块最轻边，或作为其他最小生成树算法的收缩阶段；普通一次性边表通常仍优先使用更短的 Kruskal。

以下实现显式依赖 5.5.001 Kruskal 已定义的 DSU 与 MEdge，不重复实现并查集和边记录：

```
1 optional<pair<int, vector<MEdge>>> boruvka(int n, const vector<MEdge> &e)
2 {
3     // n 是点数, e 是无向边集; 返回 MST 总权与所选边, 不连通时返回空。
4     assert(n >= 0); // 调试检查, 可删
5     DSU d(n);
6     int cnt = n, ans = 0;
7     vector<MEdge> use;
8     while (cnt > 1)
9     {
10         vector<int> best(n, -1);
11         for (int i = 0; i < (int)e.size(); i++)
12         {
13             int x = d.find(e[i].x), y = d.find(e[i].y);
14             if (x == y)
15             {
16                 continue;
17             }
18             // 求最大生成树时, 下面两处 < 要同时改成 >, 其余逻辑不变。
19             if (best[x] == -1 || e[i].w < e[best[x]].w)
20             {
21                 best[x] = i;
22             }
23             if (best[y] == -1 || e[i].w < e[best[y]].w)
24             {
25                 best[y] = i;
26             }
27         }
28         bool upd = false;
29         for (int x = 0; x < n; x++)
30         {
31             if (d.find(x) != x || best[x] == -1)
32             {
33                 continue;
34             }
35             auto v = e[best[x]];
36             if (d.merge(v.x, v.y))
37             {
38                 ans += v.w;
39                 use.push_back(v);
40                 cnt--;
41                 upd = true;
42             }
43         }
44     }
```

```
43     }
44     if (!upd)
45     {
46         return nullopt;
47     }
48 }
49 return pair{ans, use};
50 }
```

调用 `boruvka(n,e)`，成功时返回最小生成树总权和按加入顺序排列的  $n - 1$  条边，图不连通时返回空； $n = 0$  或  $n = 1$  时返回权和 0 的空树。输入是无向边表，每条边只需出现一次，允许负权、重边和自环，端点必须落在  $[0, n)$ ，总权须能由 64 位 `int` 表示。每轮扫描全部  $m$  条边并做并查集操作，共  $O(m \log n \alpha(n))$  时间、 $O(n)$  额外空间。求最大生成树时只需把代码中两处候选比较的 `<` 同时改为 `>`，无穷单位元和其余分支都不变；只改一处会破坏各连通块选择方向的一致性。已有边表且排序方便时使用 5.5.001 Kruskal，已有邻接表或矩阵时使用 5.5.013 Prim。

下面检查负权、重边和生成树恢复：

```
1 signed main()
2 {
3     vector<MEdge> e{{0, 1, 4}, {1, 2, -2}, {2, 3, 5},
4                     {0, 3, 10}, {0, 2, 1}, {0, 2, 8}};
5     auto ans = boruvka(4, e);
6     assert(ans.has_value());
7     assert(ans->first == 4 && ans->second.size() == 3);
8 }
```